

Mock Final Exam 1

(5:30 PM – 8:00 PM : 150 Minutes)

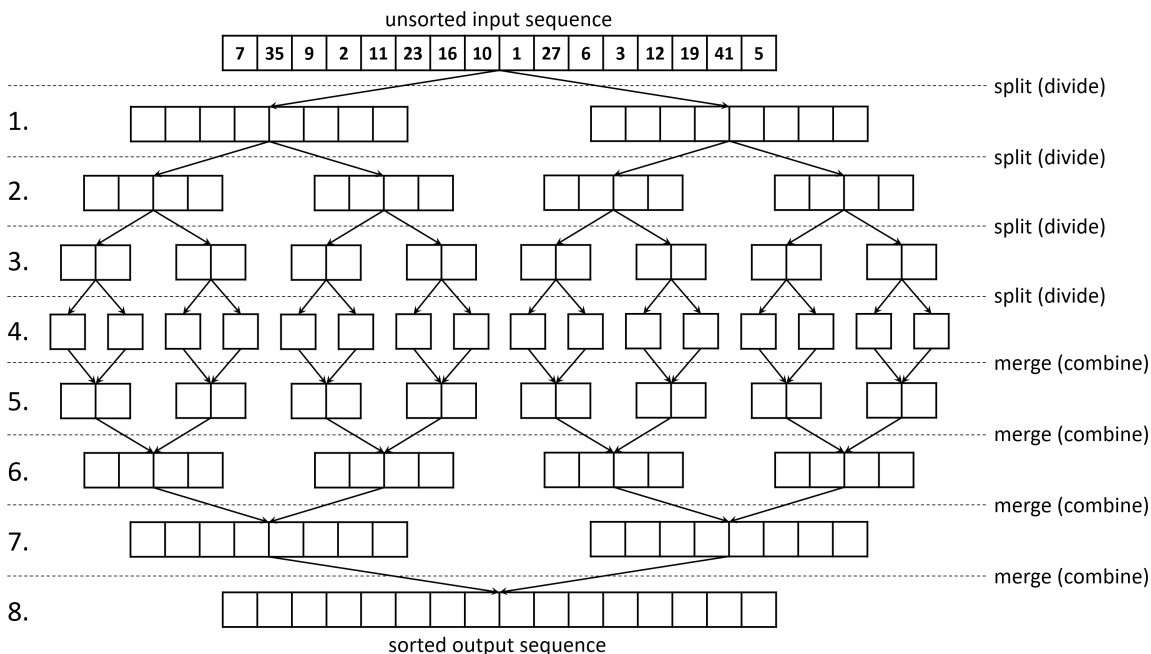
- This exam will account for 45% of your overall grade.
- There are six (6) questions, worth 250 points in total. Please answer all of them.
- This is a *closed book, closed notes* exam. *No cheat sheets* are allowed.
- You are allowed to *use scratch papers* for your calculations.
- You are *not allowed to use your own calculator*. A scientific calculator will be available inside the Respondus Lockdown Browser.

GOOD LUCK!

Question	Parts	Points
1. Mergesort	–	$2.5 \times 8 = 20$
2. Heap Operations	$(a)-(h)$	$5 \times 8 = 40$
3. Dijkstra's SSSP	$(a)-(l)$	$5 + 5 \times 10 + 10 = 65$
4. Kruskal's MST	$(a)-(j)$	$5 \times 10 = 50$
5. Longest Common Subsequence	$(a)-(c)$	$20 + 10 + 10 = 40$
6. Complexities	$(a)-(g)$	$7 \times 5 = 35$
Total		250

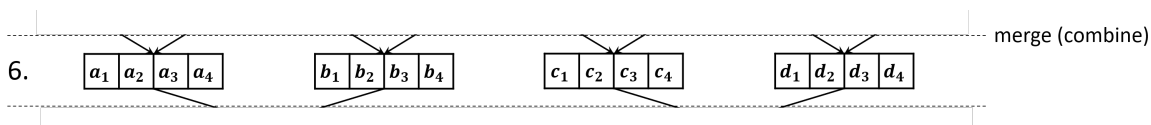
QUESTION 1. [20 Points] Mergesort. Recall that the mergesort algorithm we saw in the class (before midterm) works by splitting the given sequence at the midpoint, recursively sorting the two parts, and then merging the two sorted parts into a single sorted sequence.

This task asks you to fill out the following figure in order to show how the splitting and merging steps work for the given input sequence.



You do not need to draw the figure in your answer. You simply need to write down the contents of the cells after each split/merge step. Here is an example.

Suppose the contents of the four blocks in step 6 are as follows.



Then you write the following line for step 6 in your answer.

6. $[a_1, a_2, a_3, a_4]$ $[b_1, b_2, b_3, b_4]$ $[c_1, c_2, c_3, c_4]$ $[d_1, d_2, d_3, d_4]$

Your answer must include a separate line for each of the 8 steps.

QUESTION 2. [40 Points] Heap Operations. Consider a max-heap H that supports the following operations.

MAKE-HEAP(H): Create an empty max-heap H .

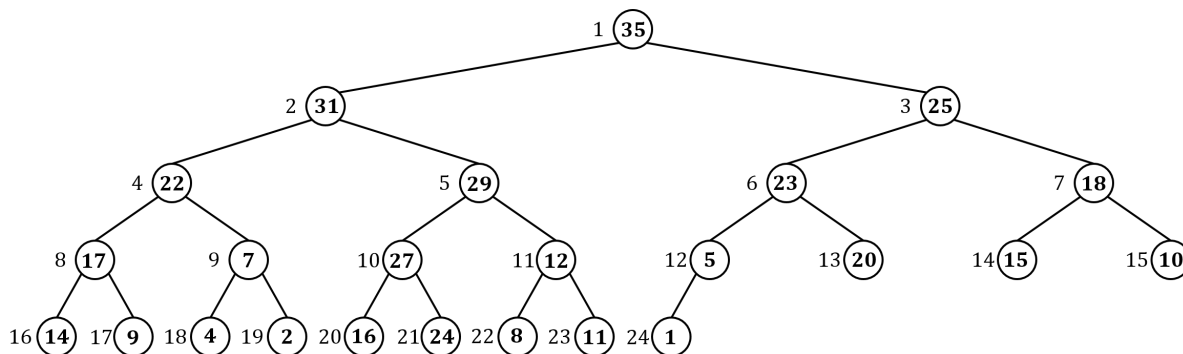
INSERT(H, k): Insert an item with key k into H .

INCREASE-KEY(H, i, k): Increase the key of the item at location i to k assuming $k \geq$ current key of the item.

FIND-MAX(H): Return the item in H containing the largest key.

EXTRACT-MAX(H): Delete an item with the largest key from H and return it.

Now consider the following max-heap H on 24 items. The number inside each node is the key of an item and the number outside is the current index of that item in the heap.



The state of the heap above in array format can be written down as follows, where for $1 \leq i \leq 24$, the key at index i of the heap is written as the i -th entry of the array:

35, 31, 25, 22, 29, 23, 18, 17, 7, 27, 12, 5, 20, 15, 10, 14, 9, 4, 2, 16, 24, 8, 11, 1

Now show the state of the heap in array format after each of the following operations assuming that they are performed in the order given below.

2(a) [5 Points] INSERT(H , 28):

2(b) [5 Points] EXTRACT-MAX(H):

2(c) [5 Points] INCREASE-KEY(H , 19, 30):

2(d) [5 Points] EXTRACT-MAX(H):

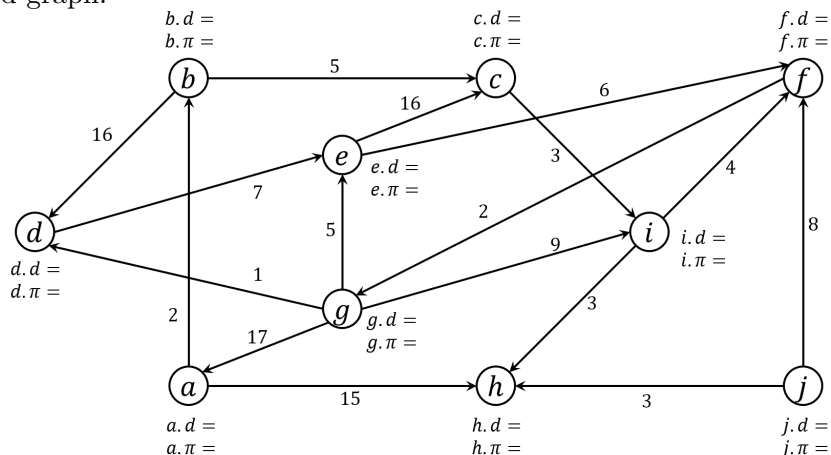
2(e) [5 Points] EXTRACT-MAX(H):

2(f) [5 Points] EXTRACT-MAX(H):

2(g) [5 Points] INSERT(H , 26):

2(h) [5 Points] EXTRACT-MAX(H):

QUESTION 3. [65 Points] Dijkstra's SSSP. Recall Dijkstra's Single-Source Shortest Paths (SSSP) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds shortest paths from a given source vertex to all other vertices of the following weighted directed graph.

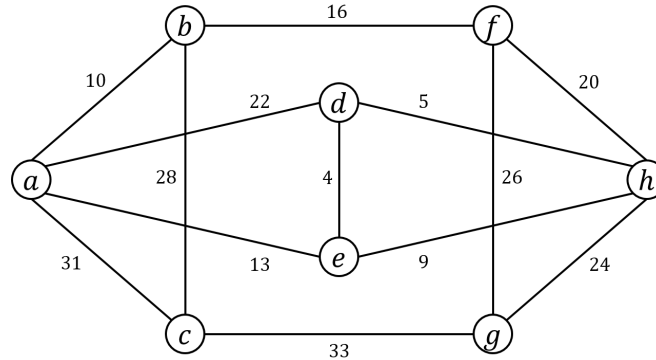


We will treat vertex a as the given source vertex. Recall that the algorithm first initializes the d and π fields of each node of the graph, and then in each step finalizes the d value of a suitable vertex and relaxes the outgoing edges of the vertex.

In your answers you can write “ π ” instead of π .

- 3(a) [5 Points] INITIALIZATION: What are the initial values of the d and π fields of each vertex?
- 3(b) [5 Points] STEP 1: Write down the 1st vertex to be finalized and the vertices whose d and π fields change (along with their new d and π values) as a result of relaxation.
- 3(c) [5 Points] STEP 2: Repeat 3(b) for the 2nd vertex to be finalized.
- 3(d) [5 Points] STEP 3: Repeat 3(b) for the 3rd vertex to be finalized.
- 3(e) [5 Points] STEP 4: Repeat 3(b) for the 4th vertex to be finalized.
- 3(f) [5 Points] STEP 5: Repeat 3(b) for the 5th vertex to be finalized.
- 3(g) [5 Points] STEP 6: Repeat 3(b) for the 6th vertex to be finalized.
- 3(h) [5 Points] STEP 7: Repeat 3(b) for the 7th vertex to be finalized.
- 3(i) [5 Points] STEP 8: Repeat 3(b) for the 8th vertex to be finalized.
- 3(j) [5 Points] STEP 9: Repeat 3(b) for the 9th vertex to be finalized.
- 3(k) [5 Points] STEP 10: Repeat 3(b) for the 10th vertex to be finalized.
- 3(l) [10 Points] By examining the final d and π fields of the vertices of the graph, write down the shortest distance and the corresponding shortest path from the source vertex (i.e., vertex a) to each of the following vertices: d , e , f , h and j .

QUESTION 4. [50 Points] Kruskal's MST. Recall Kruskal's Minimum Spanning Tree (MST) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds an MST of the following weighted undirected graph.



Recall that the algorithm examines the edges one by one in a predetermined sequence. In each step the algorithm examines one edge, and decides to either include it in the MST or discard it.

- 4(a) [5 Points] Give the order in which the algorithm will examine the edges of the graph.
- 4(b) [5 Points] In Kruskal's algorithm when do you decide not to include an edge to the MST?
- 4(c) [5 Points] Write down the 1st edge to be included in the MST.
- 4(d) [5 Points] Write down the 2nd edge to be included in the MST.
- 4(e) [5 Points] Write down the 3rd edge to be included in the MST.
- 4(f) [5 Points] Write down the 4th edge to be included in the MST.
- 4(g) [5 Points] Write down the 5th edge to be included in the MST.
- 4(h) [5 Points] Write down the 6th edge to be included in the MST.
- 4(i) [5 Points] Write down the 7th edge to be included in the MST.
- 4(j) [5 Points] What is the weight of the MST you have just computed?

QUESTION 5. [40 Points] Longest Common Subsequence. This task asks you to find the longest common subsequence (LCS) between two given sequences using the dynamic programming algorithm we saw in the class. We will find the LCS of *CGTGACG* and *GATCGCG*.

You will have to fill out the following dynamic programming table as we did in the class. For each cell in the table you will have to compute the value of the cell and choose an appropriate arrow (up: $\uparrow$, left: $\leftarrow$, up-left: $\nwarrow$) to point to the neighboring cell from which the value of the current cell has been computed.

		j	0	1	2	3	4	5	6	7
i		y_j	G	A	T	C	G	C	G	
0	x_i									
1	C									
2	G									
3	T									
4	G									
5	A									
6	C									
7	G									

5(a) [20 Points] Fill out the table given above, and report the length of an LCS.

You do not need to draw the table in your answer. Simply write one line for each row of the table. For example, suppose row 4 of the table looks like the following:

4	G		$\nwarrow$		$\uparrow$	$\nwarrow$		$\nwarrow$
		0	1	$\leftarrow$ 1	2	$\leftarrow$ 2	3	$\leftarrow$ 3

Then the line corresponding to row 4 in your answer should look like the following (U: up arrow, L: left arrow, UL: up-left arrow):

4 : 0, 1 (UL), 1 (L), 2 (U), 2 (L), 3 (UL), 3 (L), 3 (UL)

5(b) [10 Points] What is the LCS computed by the algorithm? Explain how you find it from the table.

5(c) [10 Points] From the table you computed in part (a) also find an LCS between each of the following pairs: (i) *CGTGACG* and *GATCGC*, and (ii) *CGTG* and *GATCGCG*.

QUESTION 6. [35 Points] Complexities. State the running times for the following algorithms/problems/data structural operations. If the problem is NP-Complete, NP-Hard, or has no known polynomial time algorithm just write “Hard” instead of stating the running time.

No justifications necessary, but you must state what each variable in your stated complexity means (e.g., meanings of m and n in $\mathcal{O}((m+n)\log\log m)$).

6(a) [5 Points] Prim’s MST.

6(b) [5 Points] Independent Set.

6(c) [5 Points] EXTRACT-MAX in a Binary Max-Heap.

6(d) [5 Points] Karatsuba’s Integer Multiplication Algorithm.

6(e) [5 Points] Matrix Chain Multiplication.

6(f) [5 Points] Traveling Salesman Problem.

6(g) [5 Points] Topological Sort.